



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`; ELSE `move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

Exponential Table Growth: A table-driven agent requires mapping every possible sequence of percepts (percept history) to an action. In a complex environment like Chess, the number of possible states/percept sequences is astronomically large (ex: Game Tree Complexity of chess is around 10^{120}).

Physical Constraints: Storing such a massive table exceeds the physical memory and lookup limits of any existing or future supercomputer.

As time passes (T increases), the length of the percept sequence grows linearly, but the total number of entries required in the table grows exponentially. This leads to an "Exhaustion of Physical Memory" (Memory Space Overhead) and severe lookup latency, rendering the agent completely impractical.

2. (Understand) Look at the code you wrote for your

SimpleReflexAgent . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

In Lecture 02, Condition-Action Rules are defined as IF-THEN rules that map a current state/percept directly to a specific action, ignoring any prior percept history (if condition then action).

In my SimpleReflexAgent code, this theory is directly implemented through conditional logic based solely on immediate sensor inputs (food_here and wall_ahead):

```
# Extract current percepts (Sensors)
food_here = percept.get('food_here', False)
wall_ahead = percept.get('wall_ahead', False)

# Condition-Action Rules
if food_here:
    return 'suck'      # Rule 1: IF food is present THEN suck
elif wall_ahead:
    return 'Left'      # Rule 2: IF wall is ahead THEN turn Left
else:
    return 'Up'        # Rule 3: IF path is clear THEN move Up
```

Condition: Evaluated dynamically from the percept dictionary (e.g., food_here == True).

Action: Returned immediately as a response ('suck', 'Left', or 'Up').

Pure Reflex Behavior: The agent bases its action entirely on what it senses right now, with zero memory of previous actions or states.

3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this

happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The failure occurred because a Simple Reflex Agent relies exclusively on current sensor inputs, leaving it vulnerable to infinite loops in partially observable environments.

Partial Observability: The agent cannot see the full grid or its coordinates (x,y). It only senses immediate adjacent cells (ex: wall_ahead). Because multiple different grid locations can produce the exact same percept, the agent cannot distinguish where it is.

Lack of Percept History: Without memory of past actions or previous percepts, the agent cannot recognize that it has encountered the exact same scenario repeatedly.

The Infinite Loop: When hitting a corner or wall, the agent executes a static response (ex: turning Left). Upon moving, it receives the same percept again in the next state, executing the same action over and over without escaping.

//

4. (Evaluate) In Step 1.3, you added an internal state to your ModelBasedAgent . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

In ModelBasedAgent, the internal state updates allow the agent to track hidden aspects of the world and resolve partial observability:

Sensor Model (How state reflects percepts):

The agent processes incoming raw sensor inputs via `percept.get('wall_ahead')` and `percept.get('food_here')` to construct a view of its immediate environment.

Transition Model (How the world evolves over time):

The agent tracks how its previous decisions alter its state using the internal variable `self.last_action`.

State Update Logic in Code:

When facing a wall (`wall_ahead == True`), the code evaluates `self.last_action` to cycle dynamically through actions (Left -> Down -> Right -> Up) instead of repeating the same decision. Updating `self.last_action = action` at the end of each turn updates the agent's internal model of how its state has evolved.

//

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING